OS-Orange Problem

PES2UG24CS032

1a)

```
aditya@PES2UG24CS032:~$ cd PES2UG24CS032-pes-vcs
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

1b)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

2a)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

2b)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ find .pes/objects -type f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/d6/689cc0475de79695ffc81d72886be46ee435df89b92c4e01049f636c4125fb
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/4f/6289d48e563fb8942aff1f6dd8dc1054a434e40277418e8e48b02dfe8675a9
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de6107018335 7d6968b9
.pes/objects/7b/656300ad4ee4d5aa50226a58d0055496b851300309de1c9c1afd51e896a033
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ xxd .pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc | head -20
00000000: 626c 6f62 2038 0047 6f6f 6462 7965 0a    blob 8.Goodbye.
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

3a)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ make pes
make: 'pes' is up to date.
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
Initialized empty PES repository in .pes/
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ ./pes add file1.txt file2.txt
./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  PES2UG24CS032-OS-OrangeProblem.pdf
  untracked:  README.md
  untracked:  tree.h
  untracked:  pes.c
  untracked:  pes.h
  untracked:  analysis questions.txt
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  bye.txt
  untracked:  index.h
  untracked:  object.c
  untracked:  test_tree
  untracked:  index.c
  untracked:  tree.c
  untracked:  Makefile
  untracked:  hello.txt
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  test_objects
```

3b)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776447356 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776447356 6 file2.txt
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

4a)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ ./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Initialized empty PES repository in .pes/
Committed: d6689cc0475d... Initial commit
Committed: 7b656300ad4e... Add world
Committed: 4f6289d48e56... Add farewell
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ ./pes log
commit 4f6289d48e563fb8942aff1f6dd8dc1054a434e40277418e8e48b02dfe8675a9
Author: PES User <pes@localhost>
Date:   1776447431

    Add farewell

commit 7b656300ad4ee4d5aa50226a58d0055496b851300309de1c9c1afd51e896a033
Author: PES User <pes@localhost>
Date:   1776447431

    Add world

commit d6689cc0475de79695ffc81d72886be46ee435df89b92c4e01049f636c4125fb
Author: PES User <pes@localhost>
Date:   1776447431

    Initial commit

aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

4b)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/4f/6289d48e563fb8942aff1f6dd8dc1054a434e40277418e8e48b02dfe8675a9
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7b/656300ad4ee4d5aa50226a58d0055496b851300309de1c9c1afd51e896a033
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/d6/689cc0475de79695ffc81d72886be46ee435df89b92c4e01049f636c4125fb
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

4c)

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
4f6289d48e563fb8942aff1f6dd8dc1054a434e40277418e8e48b02dfe8675a9
ref: refs/heads/main
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

Final Integraton

```
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 6cafade2da6a... Initial commit

Log after first commit:
commit 6cafade2da6ae99b62fb4749ea38698ba3e77ecc4f9c1490d785bffaeff32b85
Author: PES User <pes@localhost>
Date:   1776447545

    Initial commit
```

```
--- Second Commit ---
Committed: 61c80870976b... Update file.txt

--- Third Commit ---
Committed: aca71d45f29c... Add farewell

--- Full History ---
commit aca71d45f29c16b13cd55c4a38872d7921a7e7e351bea5801b52f0bf7a0163ec
Author: PES User <pes@localhost>
Date:   1776447545

    Add farewell

commit 61c80870976b07ee5cd868d1705cd029b695fe98385a301ad1137a1a3f66a5a5
Author: PES User <pes@localhost>
Date:   1776447545

    Update file.txt

commit 6cafade2da6ae99b62fb4749ea38698ba3e77ecc4f9c1490d785bffaeff32b85
Author: PES User <pes@localhost>
Date:   1776447545

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
aca71d45f29c16b13cd55c4a38872d7921a7e7e351bea5801b52f0bf7a0163ec
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/61/c80870976b07ee5cd868d1705cd029b695fe98385a301ad1137a1a3f66a5a5
.pes/objects/6c/afade2da6ae99b62fb4749ea38698ba3e77ecc4f9c1490d785bffaeff32b85
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/ac/a71d45f29c16b13cd55c4a38872d7921a7e7e351bea5801b52f0bf7a0163ec
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
aditya@PES2UG24CS032:~/PES2UG24CS032-pes-vcs$
```

Analysis Questions

Phase 5: Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> -- what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A branch can be represented as a file inside .pes/refs/heads/. The file stores the commit hash at the tip of that branch. To implement pes checkout <branch>, the program would first check that .pes/refs/heads/<branch> exists. It would then read the commit hash stored in that branch file.

The main metadata file that must change is .pes/HEAD. If the user checks out a branch named feature, .pes/HEAD should contain something like:

ref: refs/heads/feature

That means future commits will advance the feature branch.

However, checkout is not only a HEAD change. The working directory must also be updated to match the tree stored in the target branch's latest commit. The checkout operation needs to:

1. Read the target branch file to get the target commit hash.
2. Read the target commit object.
3. Read the root tree hash from the commit.
4. Recursively expand the tree into paths and blob hashes.
5. Compare the current index and working directory with the target tree.
6. Refuse checkout if local uncommitted changes would be overwritten.
7. Delete tracked files that exist in the current branch but not in the target branch.
8. Write files from the target tree into the working directory.
9. Replace the index with entries matching the target tree.
10. Update .pes/HEAD to point to the selected branch.

This operation is complex because it must protect user data. Changing HEAD is easy, but safely changing the working directory is hard. Checkout must handle modified files, deleted files, files added on one branch but not another, nested directories, untracked files that might be overwritten, and the index state.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

The index represents the clean baseline of the current checkout. For each tracked file, the index stores the blob hash that the working directory should match if it is clean.

To detect a dirty working directory conflict, compare three versions of each path:

index_blob: the blob hash recorded in the current index.

worktree_blob: the blob hash computed from the current file contents in the working directory.

target_blob: the blob hash for that path in the target branch's tree.

A tracked file has uncommitted changes when:

worktree_blob != index_blob

The target branch wants to change that file when:

target_blob != index_blob

Therefore, checkout must refuse when both are true:

worktree_blob != index_blob and target_blob != index_blob

That means the user has changed the file locally, and switching branches would replace that same file with a different version from the target branch.

The same idea applies to deletions. If a file is tracked in the index but missing from the working directory, that is a local change. If the target branch would also change or delete that file, checkout should refuse. Also, if the target branch needs to write a tracked file path but an untracked file already exists there, checkout should refuse to avoid overwriting untracked data.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Normally, .pes/HEAD points to a branch reference, such as:

ref: refs/heads/main

In detached HEAD state, .pes/HEAD contains a commit hash directly instead:

3f2a9c...

This means the repository is checked out at a specific commit, but not through a branch name.

If the user makes commits in detached HEAD state, the new commits are valid commit objects. HEAD moves forward to the new commit hash directly. However, no branch file in .pes/refs/heads/ is updated.

The risk is that when the user later checks out a branch, HEAD will point back to a branch reference. The commits made while detached may no longer be reachable from any branch. They are not immediately invalid, but they can eventually become garbage if nothing references them.

The user can recover those commits by creating a branch that points to the detached commit before leaving detached HEAD:

pes branch saved-work

That would create a file like:

.pes/refs/heads/saved-work

containing the detached commit hash.

If the user already left detached HEAD, recovery requires finding the commit hash. Real Git supports this with the reflog, which records previous HEAD positions. A simple VCS could support recovery by keeping a similar HEAD log, or the user could recover the commit if they had manually recorded its hash.

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects -- blobs, trees, or commits that no branch points to, directly or transitively. Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Garbage collection can be implemented as a graph traversal.

The root set is every live reference:

.pes/refs/heads/*

.pes/HEAD, if HEAD is detached

Each branch points to a commit. Starting from those commits, the algorithm marks every object reachable from them:

1. Add all branch head commit hashes to a stack.
2. Pop a hash from the stack.
3. If it has already been marked reachable, skip it.
4. Otherwise, add it to the reachable set.
5. Read the object from the object store.
6. If it is a commit, push its parent commit hashes and root tree hash.
7. If it is a tree, push every child tree hash and blob hash.

8. If it is a blob, no further traversal is needed.

After traversal, scan the entire object store. Any object hash not in the reachable set is unreachable and can be deleted.

The best data structure for tracking reachable hashes is a hash set. A hash set gives average O(1) insertion and lookup, which is important when checking many object files.

The traversal cost is:

O(number of reachable commits + reachable trees + reachable blobs)

For a repository with 100,000 commits and 50 branches, the algorithm does not visit 100,000 commits per branch, because branches usually share history. Each reachable object is visited once because the reachable set prevents repeated work.

At minimum, if all commits are reachable, GC visits about 100,000 commit objects. It also visits all reachable tree and blob objects. In a real repository, that may mean several hundred thousand or millions of total objects, depending on how many files changed and how many tree objects were created. The important estimate is based on the total number of unique reachable objects, not the number of branches multiplied by the number of commits.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Garbage collection is dangerous during a commit because creating a commit is not a single atomic action. A commit operation usually writes objects first and updates the branch reference last.

A commit may do this:

1. Write new blob objects for changed files.
2. Write new tree objects that reference those blobs.
3. Write a new commit object that references the root tree and parent commit.
4. Update the current branch reference to point to the new commit.

During the early steps, the new objects exist in the object store, but no branch points to the new commit yet. From GC's point of view, those objects may appear unreachable.

A race condition could happen like this:

1. The commit process writes blob B.
2. The commit process writes tree T that references blob B.
3. GC starts and reads all branch references.
4. The new commit has not been written or referenced yet, so GC does not mark B or T as reachable.
5. GC scans the object store and deletes B and T as unreachable.
6. The commit process writes commit C referencing tree T.
7. The commit process updates the branch to point to C.

Now the branch points to commit C, but C references tree T, which was deleted. The repository is corrupted.

Real Git avoids this with several protections. It uses lock files when updating references. It also avoids immediately pruning very new loose objects, so objects created by another process are given a grace period. Git also has reflogs, which keep recently referenced commits reachable for garbage collection purposes. Aggressive immediate pruning is possible, but it is unsafe if another Git process is running.

A simple VCS could avoid this race by using a repository-wide lock so that commit and garbage collection cannot run at the same time. A more advanced system can allow concurrency, but it must coordinate reference updates and avoid deleting recently created objects.